

ANS w Elblągu

Instytut Informatyki Stosowanej im. Krzysztofa Brzeskiego

Programowanie w językach skryptowych – laboratorium

Studium stacjonarne, sem. 4, 2025/2026

Projekt Semestralny,

Poniedziałek, grupa nr 2, 18.00-19.30

Data wykonania ćwiczenia: 27.05.2026

Data oddania sprawozdania: 08.06.2026

Nazwisko i imię: Dawid Rzepka

Nr albumu: 21299

Nazwa pliku: Raport_JobBoard_21299.pdf

Link do repozytorium projektu:

https://github.com/rzepeeeek/PROJEKT_PwJS

Link do mega:

https://mega.nz/file/W2BhVbil#ZgZyH4rGpTDPdJfCi468JHX6Z2_qE91s4wNRLHRd7DE

1. Założenie projektu:	1
2. Struktura projektu:	3
3. Opis wykorzystanych plików źródłowych	6
4. Wszystkie pliki źródłowe znajdują się na githubie	8

5. Wykorzystane technologie:.....	8
6. Struktura bazy danych	8
7. Opis funkcjonalności:	9
8. Mechanizmy bezpieczeństwa:.....	9
9. Zdjęcia z wykonanego projektu:.....	9
10. Podsumowanie.....	14

1. Założenie projektu:

Zbuduj aplikację typu job board (tablica ofert pracy) z systemem autoryzacji i rolami użytkowników.

Baza danych

Wykorzystaj następujące tabele Supabase:

profiles – tworzona automatycznie po rejestracji (trigger na auth.users): id (uuid, foreign key do auth.users), role (text, wartości: recruiter lub candidate), full_name (text), created_at (timestamp).

job_postings: id (uuid), recruiter_id (uuid, foreign key do profiles.id), company (text), position (text), location (text), salary_min (int), salary_max (int), description (text), is_active (boolean, domyślnie true), posted_at (date), created_at (timestamp).

applications: id (uuid), job_id (uuid, foreign key do job_postings.id), candidate_id (uuid, foreign key do profiles.id), applicant_name (text), applicant_email (text), cover_letter (text), status (text, wartości: pending, reviewed, rejected, accepted), applied_at (timestamp).

Autoryzacja

Utwórz strony /login i /register. Na stronie rejestracji użytkownik podaje imię i nazwisko, email, hasło oraz wybiera rolę: recruiter (rekruter) lub candidate (kandydat). Po rejestracji utwórz wpis w tabeli profiles. Zalogowany użytkownik powinien widzieć swoje imię i przycisk wylogowania w nagłówku. Niezalogowany użytkownik próbujący wejść na chronioną stronę powinien zostać przekierowany do /login.

Trasy i dostęp

/login, /register dostęp: wszyscy (niezalogowani)

/jobs dostęp: wszyscy

/jobs/:id wszyscy

/jobs/new dostęp: tylko rekruterzy

/jobs/:id/edit tylko rekruter będący autorem oferty

/jobs/:id/apply dostęp: tylko kandydaci

/dashboard dostęp: zalogowani (różny widok zależnie od roli)

Strony

/jobs – lista wszystkich aktywnych ofert pracy (`is_active = true`), posortowana po `posted_at` od najnowszych, pokazująca firmę, stanowisko, lokalizację i widełki płacowe (np. \$4000 – \$6000). Dodaj możliwość filtrowania po lokalizacji i sortowania po wynagrodzeniu. Każda oferta jest klikalna.

/jobs/:id – pełne szczegóły oferty: firma, stanowisko, lokalizacja, widełki płacowe, opis. Jeśli zalogowany użytkownik jest kandydatem i jeszcze nie aplikował na tę ofertę, wyświetl przycisk „Aplikuj”. Jeśli już aplikował, wyświetl informację o statusie jego aplikacji. Jeśli zalogowany użytkownik jest rekruterem i jest autorem tej oferty, wyświetl listę wszystkich aplikacji (imię, email, data, status) z możliwością zmiany statusu każdej aplikacji (`pending` -> `reviewed` -> `accepted` / `rejected`). Każda aplikacja ma zwijany cover letter.

/jobs/new – dostępna tylko dla rekruterów. Formularz z polami: firma, stanowisko, lokalizacja, minimalna pensja, maksymalna pensja, opis. Po zapisaniu przekierowanie do `/jobs`.

/jobs/:id/edit – dostępna tylko dla rekrutera będącego autorem oferty. Formularz identyczny jak `/jobs/new`, wypełniony aktualnymi danymi. Dodatkowo przycisk pozwalający dezaktywować ofertę (`is_active = false`), co ukrywa ją z listy.

/jobs/:id/apply – dostępna tylko dla kandydatów, którzy jeszcze nie aplikowali na tę ofertę. Na górze wyświetl stanowisko i firmę. Formularz z polami: imię i nazwisko (wstępnie wypełnione z profilu), email (wstępnie wypełnione z konta), cover letter. Po wystaniu przekierowanie do `/jobs/:id`.

/dashboard – dla rekrutera: lista jego ofert z liczbą aplikacji na każdą i możliwością przejścia do szczegółów. Dla kandydata: lista jego aplikacji z aktualnym statusem każdej z nich.

2. Struktura projektu:

projekt_job_board/

|

└─ public/

|

└─ src/

| |

| └─ assets/

| | └─ hero.png

| | └─ react.svg

| | └─ vite.svg

| |

| └─ components/

| | └─ Navbar.jsx

| | └─ ProtectedRoute.jsx

| |

| └─ pages/

| | └─ Login.jsx

| | └─ Register.jsx

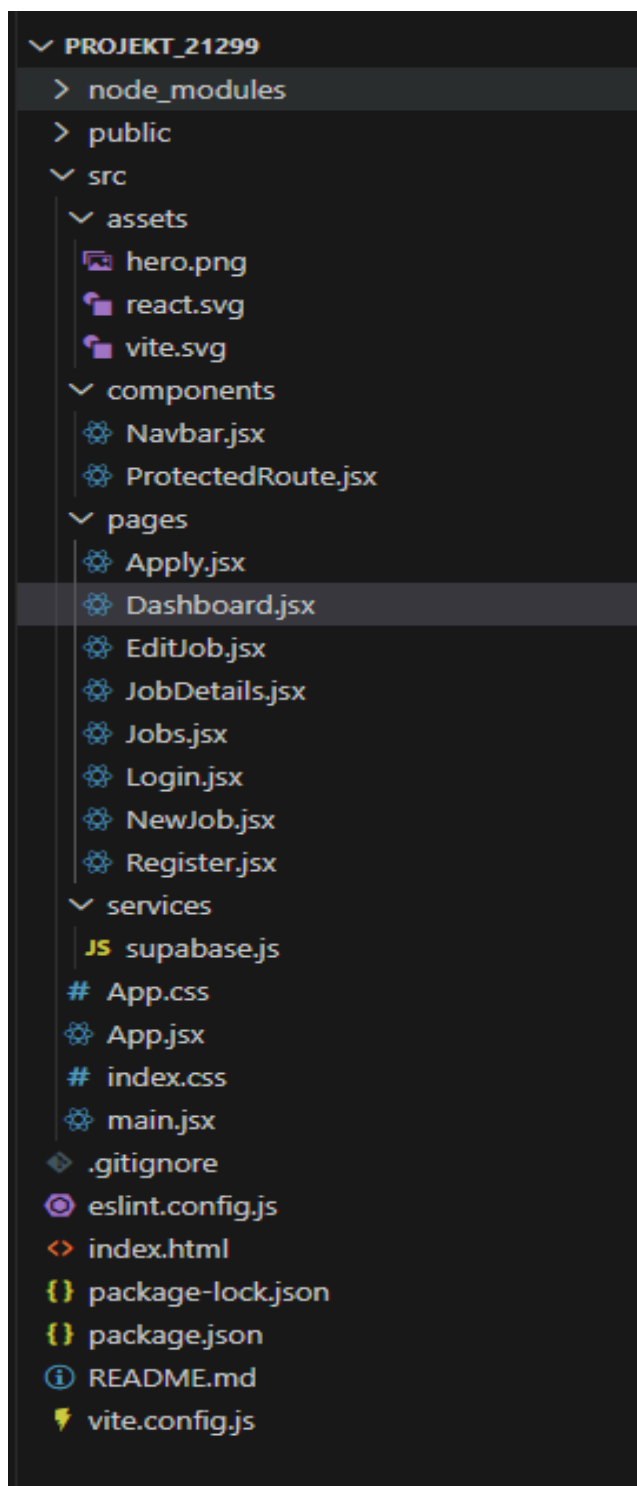
| | └─ Dashboard.jsx

| | └─ Jobs.jsx

| | └─ JobDetails.jsx

| | └─ NewJob.jsx

```
| | | └─ EditJob.jsx
| | | └─ Apply.jsx
| |
| | └─ services/
| | | └─ supabase.js
| |
| | └─ App.jsx
| | └─ App.css
| | └─ index.css
| | └─ main.jsx
|
└─ .gitignore
└─ eslint.config.js
└─ index.html
└─ package.json
└─ package-lock.json
└─ README.md
└─ vite.config.js
```



3. Opis wykorzystanych plików źródłowych

main.jsx - plik stanowi punkt wejścia aplikacji. Odpowiada za uruchomienie projektu React i wyrenderowanie głównego komponentu App.

App.jsx - zawiera konfigurację routingu aplikacji z wykorzystaniem biblioteki React Router. Definiuje wszystkie dostępne ścieżki oraz przypisane do nich komponenty.

App.css - plik zawierający główne style wykorzystywane w aplikacji, odpowiadające za wygląd poszczególnych elementów interfejsu.

index.css - plik zawiera globalne style aplikacji, takie jak wygląd tła, czcionek, przycisków, formularzy oraz kart.

services/supabase.js - odpowiada za konfigurację i nawiązanie połączenia z bazą danych Supabase. Przechowuje adres projektu oraz klucz API wykorzystywany przez aplikację

components/Navbar.jsx - komponent odpowiedzialny za wyświetlanie górnego paska nawigacyjnego. Umożliwia przechodzenie pomiędzy najważniejszymi widokami aplikacji oraz wylogowanie użytkownika.

components/ProtectedRoute.jsx - komponent zabezpieczający wybrane podstrony przed dostępem osób niezalogowanych. W przypadku braku aktywnej sesji następuje przekierowanie do strony logowania.

pages/Login.jsx - realizuje proces logowania użytkownika do aplikacji z wykorzystaniem systemu autoryzacji Supabase.

pages/Register.jsx - odpowiada za rejestrację nowych użytkowników oraz wybór roli (kandydat lub rekruter).

pages/Dashboard.jsx - wyświetla panel użytkownika dostosowany do jego roli. Kandydat może przeglądać swoje aplikacje, natomiast rekruter zarządza własnymi ofertami pracy.

pages/Jobs.jsx - wyświetla listę dostępnych ofert pracy. Zawiera możliwość filtrowania według lokalizacji oraz sortowania według wysokości wynagrodzenia lub daty dodania.

pages/JobDetails.jsx - prezentuje szczegółowe informacje o wybranej ofercie pracy. Kandydat może przejść do formularza aplikacyjnego, natomiast rekruter ma dostęp do listy aplikacji i możliwości zmiany ich statusów.

pages/Apply.jsx - zawiera formularz aplikowania na ofertę pracy. Umożliwia wprowadzenie treści zgłoszenia oraz zapisanie aplikacji w bazie danych.

pages/NewJob.jsx - komponent służący do dodawania nowych ofert pracy przez użytkownika posiadającego rolę rekrutera.

pages/EditJob.jsx - umożliwia edycję wcześniej dodanej oferty pracy oraz jej dezaktywację. Dostęp do tej funkcjonalności posiada wyłącznie autor ogłoszenia.

public/ - katalog przechowujący publiczne zasoby aplikacji dostępne podczas jej działania.

assets/ - folder zawierający pliki graficzne oraz ikony wykorzystywane w interfejsie użytkownika.

package.json - plik konfiguracyjny projektu Node.js zawierający informacje o projekcie oraz listę wykorzystywanych bibliotek i zależności.

vite.config.js - plik konfiguracyjny środowisko Vite odpowiedzialne za budowanie i uruchamianie aplikacji.

4. Wszystkie pliki źródłowe znajdują się na githubie

5. Wykorzystane technologie:

Technologia	Zastosowanie
React	Interfejs użytkownika
Vite	Środowisko uruchomieniowe
Supabase	Baza danych i autoryzacja
JavaScript	Logika aplikacji
CSS	Stylizacja interfejsu
React Router	Nawigacja między stronami

6. Struktura bazy danych

public.profiles

public.applications

+

Filter by id, role, full_name... or ask AI

	id uuid	role text	full_name text	created_at timestamp	
	221e70ab-b163-48c8-95ab-1ca3f2...	recruiter	Firma Test	2026-05-31 21:27:46.479612	
	4161ec5a-298b-4bea-b3c1-3e5cf8...	candidate	Marcin Azerbejdzan	2026-05-31 21:19:23.379698	
	b6bb1607-49de-490f-879a-5642a...	candidate	Kandydat Marcin	2026-05-31 21:35:39.760138	

Zdjęcie 1: Baza danych Profiles: Przechowuje dane użytkowników aplikacji oraz ich role.

public.profiles

public.applications

public.job_postings

+

Q

Filter by id, recruiter_id, company... or ask AI

Sort

RLS policies

Role postgres

Insert

☐	☒ id uuid	☒ recruiter_id uuid	☐ company text	☐ position text	☐ location text	☐ salary_min	☐ salary_max	☐ description
☒	a74c373c-e17e-4596-928f-958caa2214a5	221e70ab-b163-48c8-95ab-1ca3f2...	Google	Frontend Developer	Warszawa	12000	18000	Pos...

Zdjecie 2: Baza danych job_postings: Przechowuje oferty pracy dodawane przez rekruterów.

public.profiles

public.applications

public.job_postings

+

Filter by id, job_id, candidate_id... or ask AI

SortRLS policiesRole postgrsv⋮🔍👉

☐	☒ id uuid	☒ job_id uuid	☒ candidate_id uuid	☐ applicant_name text	☐ applicant_email text	☐ cover_letter text
☒	95100f5c-279e-4d14-99c8-0d68eb039f1d	a74c373c-e17e-4596-928f-958caa...	b6bb1607-49de-490f-879a-5642a...	Kandydat Marcin	testowyk@gmail.com	qwdqwdqwdqwdqwdqwd qwd qwd

Zdjęcie 3: Baza danych applications: Przechowuje aplikacje kandydatów na oferty pracy.

7. Opis funkcjonalności:

Kandydat:

- Rejestracja,
- Logowanie,
- Przeglądanie ofert,
- Aplikowanie,
- Podgląd statusów aplikacji

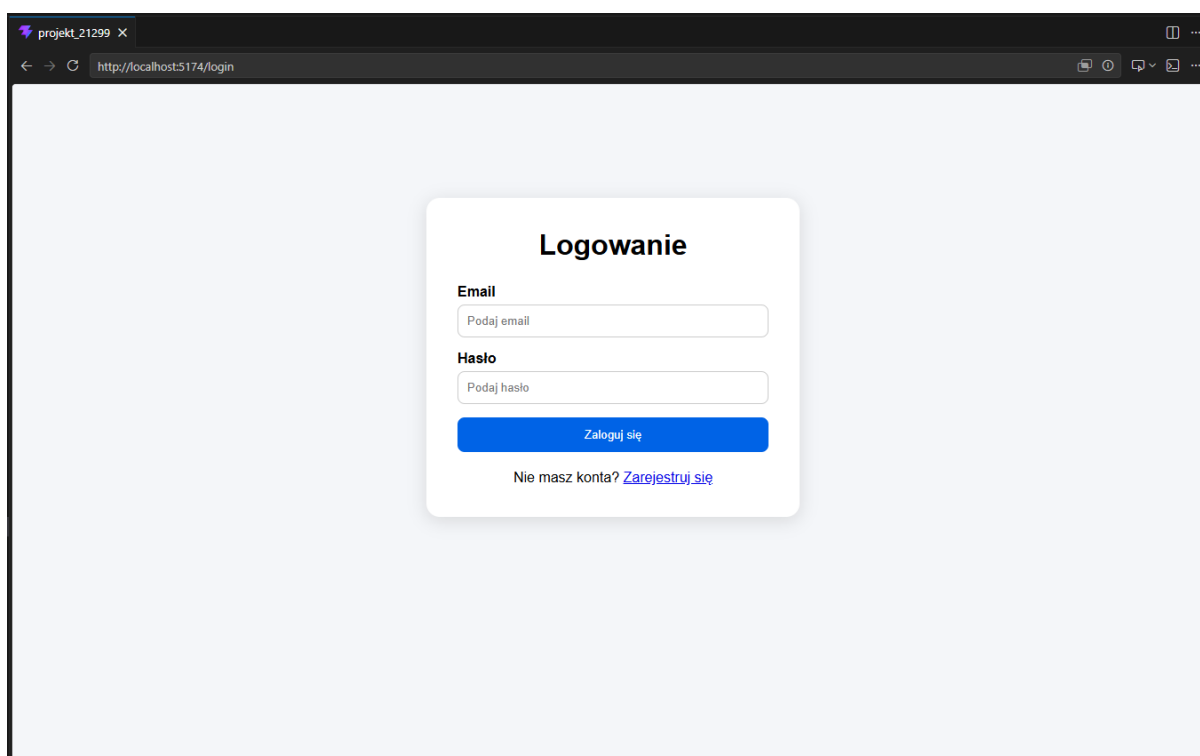
Rekruter:

- Dodawanie ofert,
- Edycja ofert,
- Dezaktywacja ofert,
- Przegląd aplikacji,
- Zamiana statusów aplikacji

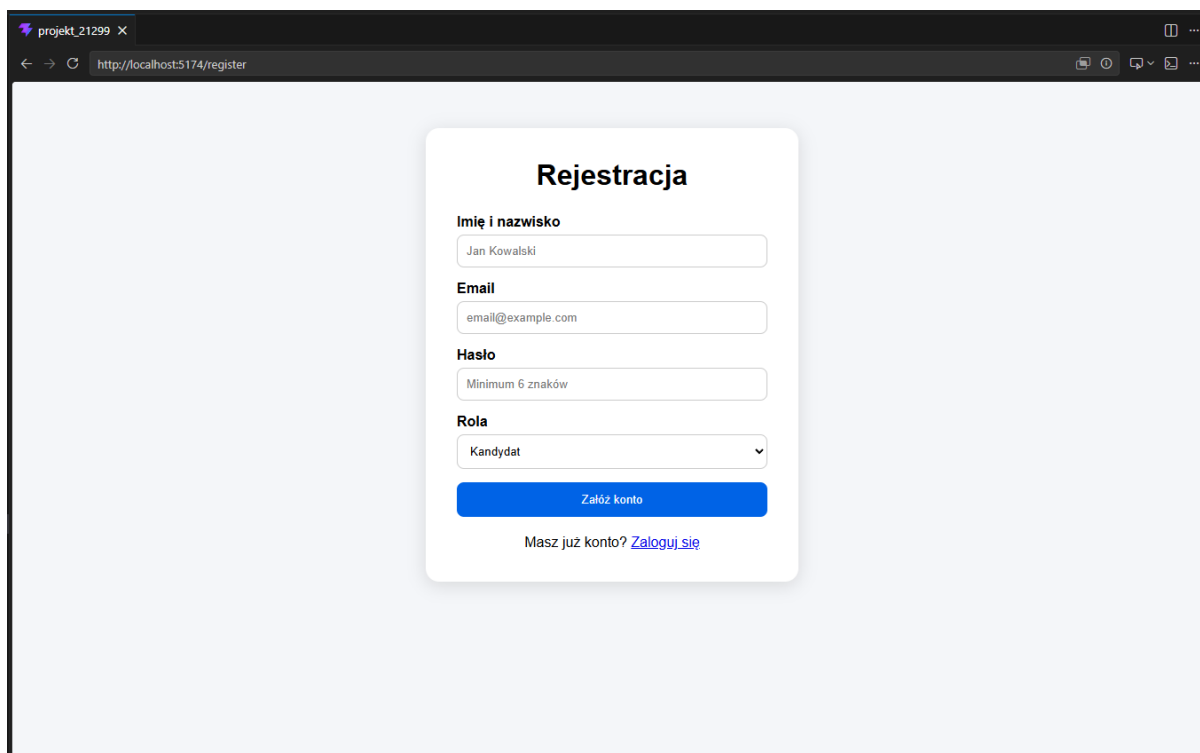
8. Mechanizmy bezpieczeństwa:

- Autoryzacja użytkowników przez Supabase Auth
- Ochrona tras za pomocą ProtectedRoute
- Ograniczenie dostępu do wybranych stron w zależności od roli użytkownika
- Blokada wielokrotnego aplikowania na tę samą ofertę
- Weryfikacja autora podczas edycji ofert pracy

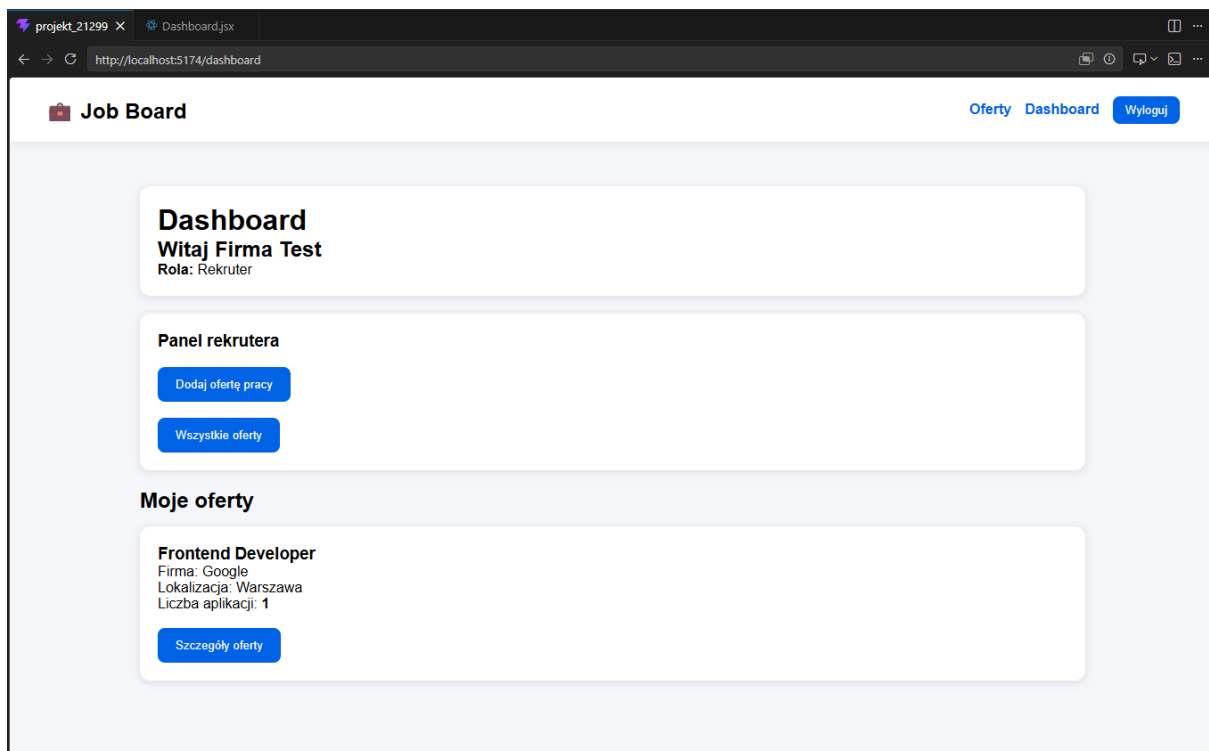
9. Zdjęcia z wykonanego projektu:



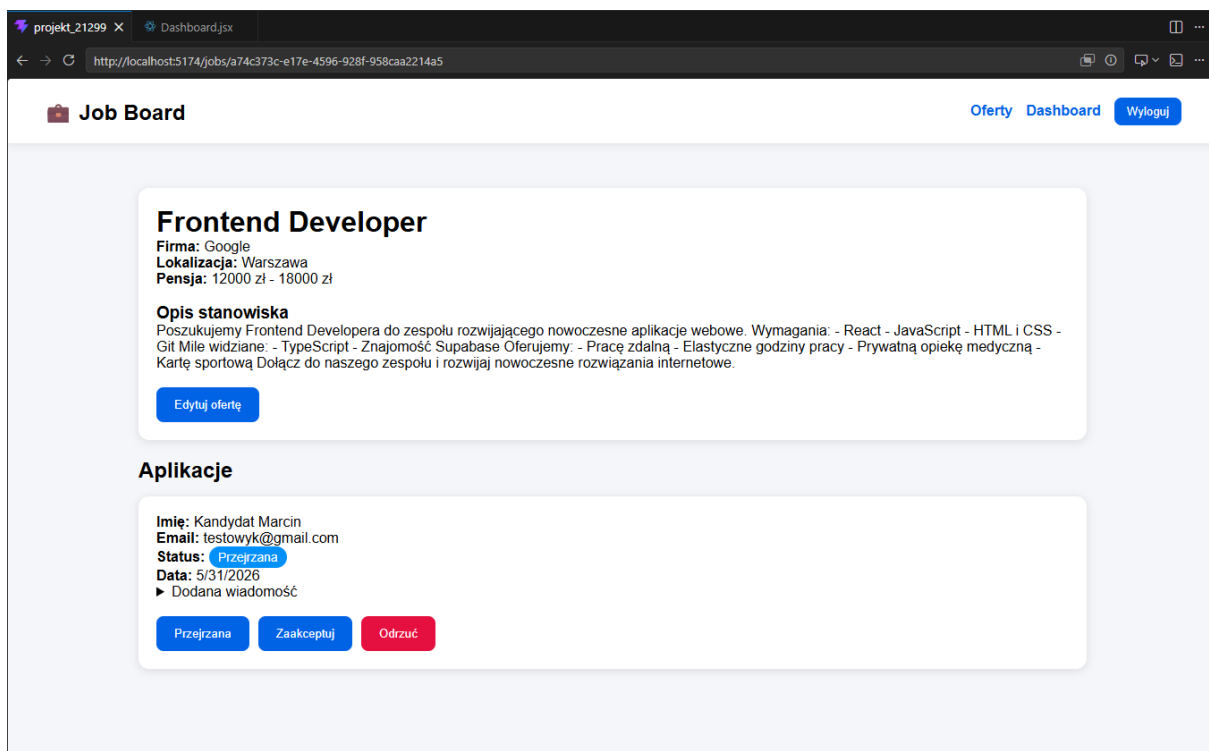
Zdjęcie 4: Strona startowa po odpaleniu aplikacji – panel logowania



Zdjęcie 5: Panel rejestracji



Zdjęcie 6: Panel Dashboard'a (na rekruterze)



Zdjecie 7: Panel przeglądania ofert wraz z aplikacjami użytkowników

Edycja oferty

Firma
Google

Stanowisko
Frontend Developer

Lokalizacja
Warszawa

Minimalna pensja
12000

Maksymalna pensja
18000

Opis stanowiska
Poszukujemy Frontend Developera do zespołu rozwijającego nowoczesne aplikacje webowe.
Wymagania:
- React
- JavaScript
- HTML i CSS
- Git
Mile widziane:
- TypeScript

Zapisz zmiany

Zdjęcie 8: Panel do edycji oferty

Job Board Oferty Dashboard Wyloguj

Oferty pracy

Lokalizacja: Wszystkie Sortowanie: Najnowsze

Frontend Developer
Firma: Google
Lokalizacja: Warszawa
Pensja: 12000 zł - 18000 zł

Zdjęcie 9: Panel wszystkich dostępnych ofert pracy

projekt_21299 X NewJob.jsx Dashboard.jsx

← → ↻ http://localhost:5174/jobs/new

Nowa oferta pracy

Firma

Stanowisko

Lokalizacja

Minimalna pensja

Maksymalna pensja

Opis stanowiska

[Dodaj ofertę](#)

Zdjęcie 10: Panel dodawania nowej oferty pracy

projekt_21299 X NewJob.jsx Dashboard.jsx

← → ↻ http://localhost:5174/dashboard

Job Board [Oferty](#) [Dashboard](#) [Wyloguj](#)

Dashboard

Witaj Kandydat Marcin
Rola: Kandydat

Panel kandydata

[Przeglądaj oferty pracy](#)

Moje aplikacje

Frontend Developer
Firma: Google
Status: [Przejrzana](#)
Data aplikacji: 5/31/2026

Zdjęcie 11: Panel kandydata

Odpalenie projektu zostało opisane w pliku README.md

10. Podsumowanie

Celem projektu było stworzenie aplikacji internetowej typu Job Board umożliwiającej publikowanie ofert pracy przez rekruterów oraz aplikowanie na nie przez kandydatów. Projekt został zrealizowany z wykorzystaniem technologii React, Vite oraz Supabase, który odpowiada za bazę danych oraz autoryzację użytkowników.

Wszystkie założone funkcjonalności zostały poprawnie zaimplementowane i przetestowane. Projekt pozwolił na praktyczne wykorzystanie wiedzy z zakresu tworzenia aplikacji webowych, obsługi baz danych oraz autoryzacji użytkowników.